WORKSHOP 2569 · วันที่ 2 จาก 2

Docker สู่ CI/CD

Compose → GitHub Actions → Deploy
push โค้ดแล้วเว็บอัปเดตเอง

github.com/taiyo130/docker-lesson-2569

ตะวัน เสริมสุข · วิศวกรรมคอมพิวเตอร์ · [11/10/2569]

ตารางวันนี้

วันที่ 2 — Compose → CI/CD

เวลา	หัวข้อ	Lab
09:00–10:30	Docker Compose + Networking	Lab 6
10:30–10:45	พักเบรก	
10:45–12:00	Workshop: web + api + db	Lab 7
12:00–13:00	พักกลางวัน	
13:00–14:30	CI: GitHub Actions → GHCR	Lab 8
14:30–14:45	พักเบรก	
14:45–16:00	CD: deploy ขึ้น stdNN.cpedev.cloud	Lab 9
16:00–17:00	Security scan, troubleshooting, Kubernetes	Lab 10

ทบทวนเมื่อวาน

เรามี image ที่ดีแล้ว 2 ตัว

`app/api/Dockerfile`

todo-api · 175 MB

multi-stage · USER node · HEALTHCHECK · รับ SIGTERM

`app/web/Dockerfile`

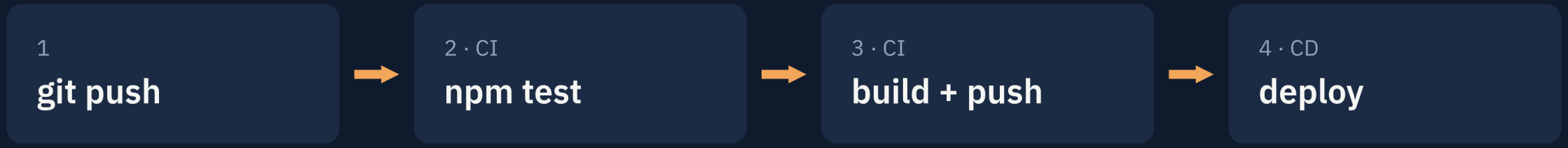
todo-web · 62 MB

Vite build → nginx · ส่ง /api ต่อให้ container ชื่อ api

ที่ยังขาด: **database** · รันทั้งชุดด้วยคำสั่งเดียว · **ส่งขึ้น server อัตโนมัติ**

เป้าหมายตอน 17:00

git push แล้วเว็บอัปเดตเอง



https://stdNN.cpedev.cloud

ผู้สอนทดสอบ pipeline นี้จริงแล้ว: push → เว็บเปลี่ยน ใช้เวลาประมาณ 2–3 นาที

LAB 6 · 09:00–10:30

Docker Compose

หลาย container · ไฟล์เดียว · คำสั่งเดียว

labs/day2/06-compose/README.md

LAB 6 · ปัญหา

รันทั้งแอปด้วย docker run = 5 คำสั่ง

```
docker network create todo-net
docker volume create pgdata
docker run -d --name db --network todo-net -e POSTGRES_PASSWORD=secret `
  -v pgdata:/var/lib/postgresql postgres:18-alpine
docker run -d --name api --network todo-net `
  -e DATABASE_URL=postgres://postgres:secret@db:5432/postgres todo-api:1.0
docker run -d --name web --network todo-net -p 8080:80 todo-web:1.0
```

ต้องจำ **ลำดับ** และ flag ทุกตัว

เพื่อนในทีมต้องพิมพ์ **ให้เหมือนเป๊ะ**

อยู่ในหัวเรา **ไม่อยู่ใน git**

LAB 6.2 · APP/COMPOSE.YAML

compose.yaml แรก: db + api

```yaml
services:
  db:
    image: postgres:18-alpine
    environment:
      POSTGRES_USER: todo
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: todo
  api:
    build: ./api
    environment:
      DATABASE_URL: postgres://todo:secret@db:5432/todo
    ports: ["3000:3000"]
    depends_on: [db]
```

service = 1 container (ชื่อ = DNS)

image: ดึงสำเร็จรูป · **build:** build จาก Dockerfile

ไม่มี **version:** แล้ว — Compose v2+ เลิกใช้

```
cd app
docker compose up -d --build
docker compose ps -a
```

LAB 6.2 · ผลที่ได้จริง

ใส่ depends_on แล้ว แต่ api ยังตาย

```
> docker compose ps -a
api: Exited (1) 7 seconds ago
db:  Up 8 seconds

> docker compose logs api
api-1 | Error: connect ECONNREFUSED 172.22.0.2:5432
api-1 |     at async PostgresStore.init (file:///app/dist/store.js:38:9)
```

depends_on ทำแค่...

เรียงลำดับการ **start** — container db ขึ้นแล้ว แต่ Postgres ยังเตรียมตัวไม่เสร็จ

แต่สังเกต...

ชื่อ **db** ถูกแปลงเป็น IP **172.22.0.2** ให้แล้ว — DNS ใช้ได้

LAB 6.3 · แก้

รอให้ db “พร้อมจริง” ด้วย healthcheck

```yaml
db:
  # ...
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U todo -d todo"]
    interval: 5s
    timeout: 3s
    retries: 10
  restart: unless-stopped
api:
  # ...
  depends_on:
    db:
      condition: service_healthy
```

pg_isready ถาม Postgres ว่ารับ connection ได้หรือยัง

api จะ start หลัง db เป็น **(healthy)** เท่านั้น

restart: unless-stopped — ตายแล้วลุกเอง (ยกเว้นเราสั่ง stop)

LAB 6.4 · NETWORKING

ชื่อ service = ชื่อ DNS

NETWORK app_default

web | api | db

Compose สร้าง network ให้ 1 วงต่อโปรเจกต์ ทุก service เรียกกันด้วยชื่อ

```
> docker compose exec api nslookup db
Name:    db
Address: 172.22.0.2

> docker network ls
app_default   bridge
```

เรียกจาก	ใช้ที่อยู่
container ใน compose เดียวกัน	http://api:3000
เครื่องเรา (browser, PowerShell)	http://localhost:3000 (ต้องมี ports:)

LAB 6.5 · .ENV

รหัสผ่านออกจากไฟล์ที่ commit

```
# app/.env (อยู่ใน .gitignore)
POSTGRES_PASSWORD=secret
```

```
# compose.yaml
POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?ตั้งค่าใน .env}
DATABASE_URL:
postgres://todo:${POSTGRES_PASSWORD}@db:5432/todo
```

เขียนแบบ	ไม่มีค่าใน .env
${VAR}	ว่าง
${VAR:-8080}	ใช้ 8080
${VAR:?ข้อความ}	หยุด + แสดงข้อความ

docker compose config แสดงไฟล์หลังแทนค่า — ใช้ตรวจก่อนรัน

```
required variable POSTGRES_PASSWORD is missing a value: ตั้งค่าใน .env
```

LAB 6 · คำสั่งที่ใช้บ่อย

docker compose ...

คำสั่ง	ทำอะไร
up -d --build	build (ถ้าจำเป็น) แล้ว start ทุก service
up -d --wait	start แล้วรอจนทุกตัว healthy
ps	สถานะ + health
logs -f api	ดู log ต่อเนื่อง (Ctrl+C ออก)
exec api sh	เข้าไปใน container
config	ดูไฟล์หลังแทนค่า .env
down	ลบ container + network · volume ยังอยู่
down -v	ลบ volume ด้วย → ข้อมูลหาย

LAB 7 · 10:45–12:00

web + api + db

ทั้งแอปขึ้นด้วยคำสั่งเดียว · ข้อมูลไม่หาย · พร้อมขึ้น CI/CD

labs/day2/07-fullstack/README.md

LAB 7.1 · ภาพรวม

3 service ใน compose.yaml เดียว




```yaml
web:
  build: ./web
  ports: ["${WEB_PORT:-8080}:80"]
  depends_on: [api]
  restart: unless-stopped
```

web ส่ง **/api/...** ไปที่ **api:3000** (ค่า default ของ API_UPSTREAM จาก Lab 5)

ลบ **ports:** ของ api ออก — เปิดสู่ภายนอกแค่ web เหมือน db

เปิดเว็บแล้วดู footer: **store: postgres** ✓

LAB 7.2 · ทดลองจริง

down vs down -v: ข้อมูลรอดหรือหาย

```
  db:
    # ...
    volumes:
      - pgdata:/var/lib/postgresql

volumes:
  pgdata:
```

หลังคำสั่ง	GET /api/todos
POST "persist me"	[{"title":"persist me"}]
down → up	[{"title":"persist me"}] ✓ ยังอยู่
down -v → up	[ ] หายหมด

Postgres 18 ใช้ path **/var/lib/postgresql** (ไม่ใช่ .../data) · บน server **อย่าใช้ down -v** เด็ดขาด ถ้าไม่ได้ตั้งใจลบ database

LAB 7.3 · เตรียมสำหรับ CI/CD

ไฟล์เดียว ใช้ได้ทั้งเครื่องเราและ server

```
api:
  build: ./api
  image: ${API_IMAGE:-todo-api}:${TAG:-latest}
  environment:
    APP_VERSION: ${TAG:-dev}  # แสดงใน footer
```

	บนเครื่องเรา	บน server (บ่ายนี้)
คำสั่ง	up -d --build	pull แล้ว up -d --no-build
image	todo-api:latest	ghcr.io/<user>/todo-cicd-api:<sha>
ใคร build	เครื่องเรา	GitHub Actions

LAB 7.4 · ภารกิจก่อนพักเที่ยง

ลองให้ครบ 5 ข้อ

1 · build แค่ตัวเดียว

```
docker compose up -d --build web
```

2 · ดู log ทุกตัวพร้อมกัน

```
docker compose logs -f
```

3 · ดูข้อมูลใน database

```
exec db psql -U todo -c "SELECT * FROM todos"
```

4 · api ล่ม เว็บเป็นยังไง?

```
docker compose stop api
```

⭐ 5 · เปิด ports ให้ db ชั่วคราวแล้วต่อด้วยโปรแกรมในเครื่อง — ทำไม production ไม่ควรเปิด?

LAB 8 · 13:00–14:30

CI ด้วย GitHub Actions

ทุก push → test → build → เก็บ image ที่ GHCR

labs/day2/08-ci/README.md

LAB 8 · แนวคิด

CI กับ CD คืออะไร

CONTINUOUS INTEGRATION

ทุก push ถูกตรวจ

test + build อัตโนมัติ — โค้ดที่พังรู้ภายในไม่กี่นาที ไม่ใช่ตอนส่งงาน

Lab 8 · ช่วง 13:00

CONTINUOUS DEPLOYMENT

ผ่านแล้วขึ้นจริง

image ที่ผ่าน CI ถูกส่งขึ้น server ให้เอง — ไม่มีใคร ssh เข้าไปแก้มือ

Lab 9 · ช่วง 14:45

Docker ทำให้ **สิ่งที่ test = สิ่งที่ deploy** — image ตัวเดียวกันทุกขั้น

LAB 8.1 · REPO ของตัวเอง

สร้าง todo-cicd แล้ว push โค้ดขึ้นไป

```
# GitHub → New repository → todo-cicd → Public → Create
cd docker-lesson-2569
robocopy app ..\todo-cicd\app /E /XD node_modules dist
cp .gitignore ..\todo-cicd\
cd ..\todo-cicd
git init -b main
git add .
git status       # ⚠️ ต้องไม่เห็น .env และ node_modules
git commit -m "Todo app with Docker Compose"
git remote add origin https://github.com/<user>/todo-cicd.git
git push -u origin main
```

Public — Actions ใช้ฟรีไม่จำกัด และ image บน GHCR pull ได้โดยไม่ต้อง login · **robocopy** มีในทุก Windows

LAB 8 · คำศัพท์

workflow → job → step

WORKFLOW · .github/workflows/ci-cd.yml

`on: push, pull_request`

job: test

step: checkout

step: npm ci

step: npm test

job: build

needs: test

step: login

step: build-push

คำ	ความหมาย
on:	เหตุการณ์ที่ทำให้รัน
job	เครื่องใหม่ 1 เครื่อง (runner) รันพร้อมกัน ถ้าไม่มี needs:
step	run: คำสั่ง shell หรือ uses: action สำเร็จรูป
needs:	รอ job อื่นผ่านก่อน

LAB 8.2 · .GITHUB/WORKFLOWS/CI-CD.YML

workflow แรก: test

```yaml
on:
  push: { branches: [main] }
  pull_request:
jobs:
  test:
    runs-on: ubuntu-latest
    defaults: { run: { working-directory: app/api } }
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v7
        with: { node-version: 24, cache: npm,
          cache-dependency-path: app/api/package-lock.json }
      - run: npm ci
      - run: npm test
      - run: npm run build
```

path ต้องเป๊ะ: **.github/workflows/**

push แล้วดูแท็บ **Actions** → คลิก job → อ่าน log ทีละ step

ของจริง: job test ใช้เวลา **9 วินาที**

LAB 8.3 · ตั้งใจทำพัง

test ไม่ผ่าน = ไม่ไปต่อ

```
// app/api/test/app.test.ts
it("reports health", async () => {
  const res = await request(createApp(new MemoryStore())).get("/api/health");
  expect(res.status).toBe(500);   // ของเดิม 200
```

push → ✗ แดง
test ล้ม · build ไม่รัน · อีเมลแจ้ง

→

อ่าน log
หาบรรทัด expected 500 · received 200

→

แก้กลับ → ✓ เขียว
push ใหม่ pipeline ไปต่อ

LAB 8.4 · JOB BUILD

build แล้ว push ขึ้น GHCR

```yaml
build:
  needs: test
  permissions: { contents: read, packages: write }
  strategy: { matrix: { service: [api, web] } }
  steps:
    - uses: actions/checkout@v7
    - id: image  # ชื่อต้องเป็นตัวเล็ก
      run: echo "name=ghcr.io/${GITHUB_REPOSITORY,,}-..." >> ...
    - uses: docker/setup-buildx-action@v4
    - uses: docker/login-action@v4  # GITHUB_TOKEN
    - uses: docker/build-push-action@v7
      with:
        context: app/${{ matrix.service }}
        tags: …:${{ github.sha }} และ …:latest
        cache-from/to: type=gha
```

needs — รอ test ผ่าน

packages: write — ให้ token push image ได้

matrix — แตกเป็น 2 job รันพร้อมกัน

ของจริง: build ครั้งแรก **~1 นาที** · มี cache **~30 วิ**

LAB 8.4 · ผลลัพธ์

image อยู่ที่ GHCR พร้อม tag 2 อัน

```
ghcr.io/taiyo130/docker-lesson-2569-demo-api:300442977462cda6c005bed5becc397fd784e201
ghcr.io/taiyo130/docker-lesson-2569-demo-api:latest
```

:<git sha>

ไม่เปลี่ยนตลอดไป — รู้ว่ามาจาก commit ไหน ใช้ deploy และ rollback

:latest

ชี้ไปตัวล่าสุดเสมอ — สะดวกตอนลองเล่น **ไม่ใช้ deploy จริง**

```
docker pull ghcr.io/<user>/todo-cicd-api:latest  # repo public → ไม่ต้อง login
```

LAB 8.5 · ⭐ PULL REQUEST

PR: build ตรวจ แต่ยังไม่ push

```
git switch -c feature/new-title
# แก้ TITLE ใน app/web/src/App.tsx
git commit -am "Change title"
git push -u origin feature/new-title
```

```
# ใน build-push-action
push: ${{ github.event_name
       == 'push' }}
```

เหตุการณ์	test	build	push + deploy
pull_request	รัน	รัน	ไม่ทำ
push เข้า main	รัน	รัน	ทำ

LAB 9 · 14:45–16:00

CD: deploy ขึ้น server

push เข้า main แล้ว stdNN.cpedev.cloud อัปเดตเอง

labs/day2/09-cd/README.md

LAB 9 · SERVER ของเรา

1 คน = 1 LXC container ที่มี Docker



2 CPU · 2 GB RAM ต่อคน · เป็น root ใน container ของตัวเองได้ (sudo) แต่ไม่กระทบเพื่อน

LAB 9.1 · เข้า SERVER ครั้งแรก

ค่าของเราทั้งหมดอยู่ในการ์ด

อะไร	ค่า	ตัวอย่าง std07
SSH	ssh -p 50NN student@76.13.177.240	-p 5007
เว็บ	https://stdNN.cpedev.cloud	std07.cpedev.cloud
API	https://stdNN.cpedev.cloud/api/info	std07.../api/info
password	ในการ์ด · เปลี่ยนด้วย passwd	

```
PS> ssh -p 50NN student@76.13.177.240  # ตอบ yes · พิมพ์ password ไม่ขึ้นตัวอักษร = ปกติ
student@stdNN:~$ docker ps
student@stdNN:~$ passwd
```

LAB 9.2 · SSH KEY

SSH key: กุญแจคู่สำหรับ GitHub

private key · ~/.ssh/deploy

เก็บเป็น **Secret SSH_KEY** ใน GitHub — ห้ามหลุด

public key · deploy.pub

วางใน **~/.ssh/authorized_keys** บน server — เปิดเผยได้

```
# รันบน server (หลัง ssh เข้าไปแล้ว)
ssh-keygen -t ed25519 -f ~/.ssh/deploy -N "" -C github-actions
cat ~/.ssh/deploy.pub >> ~/.ssh/authorized_keys
cat ~/.ssh/deploy   # คัดลอกทั้งก้อน -----BEGIN … END-----
```

LAB 9.3 · บน SERVER ครั้งเดียว

.env ของ production อยู่บน server

```
mkdir -p ~/todo
printf 'POSTGRES_PASSWORD=%s\nWEB_PORT=8080\n' "$(openssl rand -hex 16)" > ~/todo/.env
cat ~/todo/.env
```

WEB_PORT ต้องเป็น 8080

nginx ของ server ส่ง stdNN.cpedev.cloud มาที่ port 8080

รหัสสุ่มใหม่

ไม่อยู่ใน git ไม่อยู่ใน GitHub — มีที่เดียว คือ server

ทำครั้งเดียว

pipeline ส่งแค่ compose.yaml ไม่แตะ .env

LAB 9.4 · SETTINGS → SECRETS AND VARIABLES → ACTIONS

Secret 1 ตัว · Variable 4 ตัว

SECRETS

SSH_KEY

อ่านกลับไม่ได้ · ใน log เห็นเป็น ***

Variable	ค่า
DEPLOY_HOST	76.13.177.240
DEPLOY_PORT	50NN
DEPLOY_USER	student
APP_URL	https://stdNN.cpedev.cloud

ไม่ต้องตั้ง **GITHUB_TOKEN** — GitHub สร้างให้ใหม่ทุก run และหมดอายุเมื่อ run จบ

LAB 9.5 · JOB DEPLOY · ฉบับเต็มใน README

5 step จาก runner ถึงเว็บจริง

step	ทำอะไร
Setup SSH key	เขียน SSH_KEY ลงไฟล์ + ssh-keyscan จำ host key ของ server
Copy compose	เช็กว่ามี ~/todo/.env แล้ว scp compose.yaml ขึ้นไป (ไม่ส่งโค้ด)
Login GHCR	ส่ง GITHUB_TOKEN ทาง stdin ไม่ให้โผล่ใน command line
Deploy	TAG=<sha> → compose pull → up -d --no-build --wait
Smoke test	curl APP_URL/api/info แล้วเช็กว่าเห็น sha ที่เพิ่ง deploy

```
deploy:  needs: build · if: github.event_name == 'push' && vars.DEPLOY_HOST ≠ ''
```

LAB 9.6 · วงจรเต็ม

แก้ 1 บรรทัด → ขึ้น production

```
// app/web/src/App.tsx
const TITLE = "Docker Todo ของ stdNN 🚀";

git commit -am "New title"
git push    # เริ่มจับเวลา ⏱️
```

test	build api + web	deploy + smoke test	รวม
9 วิ	**~1 นาที**	**~40 วิ**	**~2 นาที**

LAB 9.7 · ROLLBACK

deploy แล้วพัง? กลับไป sha เก่า

```
# วิธี 1 — บน server (เร็วสุด ไม่ต้อง build)
cd ~/todo
TAG=<sha เก่า> API_IMAGE=ghcr.io/<user>/todo-cicd-api WEB_IMAGE=ghcr.io/<user>/todo-cicd-web \
  docker compose up -d --no-build --wait
```

วิธี 2 — GitHub

Actions → run เก่าที่เขียว → **Re-run all jobs**

วิธี 3 — git

git revert HEAD → push → pipeline deploy ให้ ประวัติครบ

ทดสอบบน std00: rollback แล้ว version เป็น sha เก่าทันที · **todo ที่เพิ่มไว้ยังอยู่** (อยู่ใน volume ไม่ใช่ใน image)

LAB 10 · 16:00–17:00

Security & ต่อยอด

สแกนช่องโหว่ใน CI · แก้ปัญหา · ไปต่อ Kubernetes

labs/day2/10-security/README.md

LAB 10.1 · TRIVY ใน JOB BUILD

image ของเรามีช่องโหว่กี่ตัว?

```
- uses: aquasecurity/trivy-action@v0.36.0
  if: github.event_name == 'push'
  with:
    image-ref: ${{ steps.image.outputs.name }}
          :${{ github.sha }}
    severity: CRITICAL,HIGH
    ignore-unfixed: true
    exit-code: '0'  # แค่รายงาน
```

4
4

web · HIGH · package ของ OS ใน nginx:1.29-alpine (curl, libssl3, libxml2...)

7

api · HIGH · ทั้งหมดอยู่ใน **npm** ที่ติดมากับ node:24-alpine

0

dependency ของแอปเราเอง (express, pg)

LAB 10.2 · แก้

บรรทัดเดียวต่อ image → เหลือ 0

web · 44 → 0

```
FROM nginx:1.29-alpine
RUN apk upgrade --no-cache
```

อัปเดต package ของ OS ที่มีเวอร์ชันแก้แล้ว

api · 7 → 0

```
# runtime stage ก่อน USER node
RUN rm -rf /usr/local/lib/node_modules/npm \
  /usr/local/bin/npm /usr/local/bin/npx
```

แอปรันด้วย node dist/server.js ไม่ใช้ npm ตอนรัน

หลักเดียวกับเมื่อวาน: ของน้อย = ช่องโหว่น้อย · multi-stage · non-root · อัปเดต base image สม่ำเสมอ

LAB 10.2 · EXIT-CODE: '1'

เจอช่องโหว่ = ไม่ deploy



demo repo หลังแก้: scan ผ่านด้วย exit-code '1' และ deploy ขึ้น std00 ✓

ข้อสังเกต: scan อยู่หลัง push → image ที่มีช่องโหว่ยังขึ้น GHCR ได้ แต่ **ไม่ถูก deploy** · ทีมจริงมัก build → scan → push

LAB 10.3 · TROUBLESHOOTING

error ที่น่าจะเจอวันนี้

ข้อความ	แก้
ECONNREFUSED ...:5432	healthcheck + condition: service_healthy
required variable POSTGRES_PASSWORD is missing	สร้าง .env ข้าง compose.yaml
port is already allocated	docker ps → ลบตัวเก่า หรือเปลี่ยน WEB_PORT
denied: permission_denied: write_package	permissions: packages: write
repository name must be lowercase	${GITHUB_REPOSITORY,,}
Permission denied (publickey)	SSH_KEY ไม่ครบก้อน / ไม่อยู่ใน authorized_keys
connect to host ... port 22: refused	ตั้ง variable DEPLOY_PORT=50NN
ไม่พบ ~/todo/.env บน server	ทำ Lab 9.3
เว็บขึ้น “ยังไม่มีแอปรันอยู่ ...”	ยังไม่ deploy หรือ WEB_PORT ไม่ใช่ 8080

LAB 10.3 · วิธีคิด

ไล่จากข้างนอกเข้าข้างใน

1
Actions log
step ไหนแดง?

2
ssh เข้า server
cd ~/todo

3
compose ps
ตัวไหนไม่ healthy? tag เป็น sha ไหน?

4
compose logs
logs --tail 50 api

5
exec เข้าไป
exec api sh · wget, env

อ่าน error **ทั้งบรรทัด** ก่อนถามเพื่อน — ส่วนใหญ่บอกคำตอบไว้แล้ว

LAB 10.4 · ต่อยอด

จาก Compose ไป Kubernetes

Compose (วันนี้)	Kubernetes
services: api	Deployment
ชื่อ service = DNS	Service
nginx uu server	Ingress
volumes: pgdata	PersistentVolumeClaim
.env	Secret / ConfigMap
healthcheck	readiness / liveness probe
compose up -d	kubectl apply -f k8s/

Compose = **เครื่องเดียว**

Kubernetes = **หลายเครื่อง**, scale อัตโนมัติ, deploy ไม่มี downtime

image ที่ CI ของเรา push ใช้ต่อได้ทันที

ลองเอง: Docker Desktop → Settings → Kubernetes → Enable

สรุป 2 วัน

จาก docker run ถึง pipeline ของตัวเอง

วันที่ 1

- container, image, layer
- Dockerfile, cache, multi-stage
- non-root, healthcheck
- volume, bind mount, env

วันที่ 2

- Compose, service DNS, healthcheck
- GitHub Actions: test → build → GHCR
- deploy ผ่าน SSH, rollback ด้วย sha
- Trivy: สแกนก่อน deploy

เก็บ repo **todo-cicd** ไว้ — ใส่ใน portfolio ได้เลย

ก่อนกลับ

Post-test

20 ข้อ · ลิงก์อยู่ใน README ของ repo และท้าย Lab 10

github.com/taiyo130/docker-lesson-2569 → 📝 แบบทดสอบ

จบการอบรม

Q & A

ขอบคุณครับ 🐳

github.com/taiyo130/docker-lesson-2569 · ตัวอย่าง: docker-lesson-2569-demo